

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

Question

1. (20 Poin)

- a) Buat dua program Python untuk menghitung kuadrat dari angka 1 hingga 1.000.000 menggunakan dua pendekatan paralel:

1. Multithreading

```
import threading
import time

def calculate_square(angka):
    return angka * angka

def multithreading_approach(x):
    threads = []
    results = [0] * len(x)
    [
        def thread_task(start_index, end_index):
            for i in range(start_index, end_index):
                results[i] = calculate_square(x[i])

    num_threads = 4 # adjustable
    chunk_size = len(x) // num_threads

    for i in range(num_threads):
        start_index = i * chunk_size
        end_index = (i + 1) * chunk_size if i != num_threads - 1
    else len(x) :
        thread = threading.Thread(target=thread_task, args=(
start_index, end_index))
        threads.append(thread)
        thread.start()

    for thread in threads:
        thread.join()

    return results

if __name__ == "__main__":
    x_to_square = list(range(1, 1_000_001))

    print("Menjalankan Multithreading...")
    start_time_threading = time.time()
    multithreading_approach(x_to_square)
    end_time_threading = time.time()
    print(f"Waktu eksekusi Multithreading: {end_time_threading -
start_time_threading:.4f} deti ")
    k"
```

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

2. Multiprocessing

```
import multiprocessing
import time

def calculate_square(angka):
    return angka * angka

def multiprocessing_approach(x):
    # Using Pool for easier distribution of tasks
    with multiprocessing.Pool(processes=multiprocessing.cpu_count
    ()) as pool:
        results = pool.map(calculate_square, x)
    return results

if __name__ == "__main__":
    x_to_square = list(range(1, 1_000_001))

    print("Menjalankan proses Multiprocessing...")
    start_time_multiprocessing = time.time()
    multiprocessing_approach(x_to_square)
    end_time_multiprocessing = time.time()
    print(f"waktu eksekusi Multiprocessing: {
    end_time_multiprocessing - start_time_multiprocessing:.4f} deti
    k")
```

b) Ukur dan bandingkan waktu eksekusi kedua program(screenshot output).

```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
(base) amaheeen@Farhans-MacBook-Pro UAS % /usr/local/bin/python3 "
py"
Menjalankan Multithreading...
Waktu eksekusi Multithreading: 0.1748 detik
(base) amaheeen@Farhans-MacBook-Pro UAS % /usr/local/bin/python3 "
py"
Menjalankan proses Multiprocessing...
waktu eksekusi Multiprocessing: 0.6812 detik
(base) amaheeen@Farhans-MacBook-Pro UAS %
```

Berdasarkan hasil percobaan di laptop saya:

- Waktu eksekusi multithreading : 0.1748 detik
- Waktu eksekusi multiprocessing : 0.6812 detik

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

Terlihat bahwa program yang menggunakan multithreading berjalan lebih cepat dibandingkan dengan multiprocessing untuk tugas perhitungan kuadrat ini.

c) Berdasarkan hasil Anda, jelaskan dampak GIL terhadap performa pada tugas CPU-bound.

Biasanya, multiprocessing lebih unggul untuk tugas CPU-bound karena bisa memanfaatkan banyak core tanpa terhalang *Global Interpreter Lock* (GIL). Namun, dalam kasus ini, multithreading justru lebih cepat. Hal ini kemungkinan disebabkan oleh *overhead* pembuatan proses pada multiprocessing yang cukup besar, sementara tugas menghitung kuadrat 1 juta angka tergolong ringan dan cepat. Akibatnya, "biaya" pembuatan proses menjadi lebih dominan dibandingkan manfaat paralelisme penuh yang ditawarkan multiprocessing.

Selain itu, karena skalanya tidak terlalu berat, dampak GIL pada multithreading tidak terlalu terasa. Pada sistem dengan jumlah core terbatas, keuntungan multiprocessing pun semakin kecil. Jadi, dalam kondisi seperti ini, multithreading lebih efisien karena lebih ringan dalam manajemen resource, meskipun secara teori GIL tetap menjadi batasan. Efektivitas masing-masing pendekatan sangat tergantung pada kompleksitas tugas dan lingkungan eksekusi.

d) Referensi:

1. Temukan 1 referensi akademik yang membahas efek GIL

Ruys, W., Lee, H., You, B., Talati, S., Park, J., Almgren-Bell, J., Yan, Y., Fernando, M., Erez, M., Gligoric, M. and Burtscher, M., 2025. "Performance Characterization of Python Runtimes for Multi-device Task Parallel Programming." International Journal of Parallel Programming, 53(2), pp.1-24.

2. Cantumkan bagian spesifik kutipan dari masing-masing referensi yang mendukung analisis Anda.

- "When it comes to Python, finer-grained multithreaded parallelism has faced a unique challenge: the *Global Interpreter Lock* (GIL) in the CPython interpreter allows only one thread at a time to execute Python bytecode, effectively serializing all work except C-extension library calls and I/O." (Ruys et al., 2023, Hal. 1)

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

- "However, many Python tasking systems adopt ③, employing multiple processes each with distinct interpreters. This introduces overheads, notably from inter-process communication and the initialization cost of loading libraries into each process." (Ruys et al., 2023, Hal. 2)

3. Jelaskan bagaimana bagian tersebut mendukung analisis hasil eksperimen Anda

Ruys et al. (2023) menjelaskan bahwa *Global Interpreter Lock* (GIL) pada *CPython* membatasi eksekusi *thread* secara paralel, sehingga multithreading sering dianggap kurang efektif untuk tugas CPU-bound. Namun, hasil eksperimen saya justru menunjukkan bahwa multithreading lebih cepat dibandingkan multiprocessing dalam menghitung kuadrat 1 juta angka. Hal ini disebabkan oleh overhead pada multiprocessing, seperti biaya komunikasi antar-proses dan inisialisasi proses baru, yang ternyata lebih berat dibandingkan hambatan GIL pada multithreading. Multithreading lebih efisien pada skala tugas ini karena thread lebih ringan dan berbagi memori, sedangkan multiprocessing memerlukan biaya tambahan yang tidak sebanding untuk tugas sederhana seperti ini.

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

2. (20 Poin)

- a) Buat program Python multithreaded dengan dua threading. Thread yang melakukan increment ke global counter sebanyak 100.000 kali per thread (tanpa sinkronisasi).

```
import threading
import time

global_counter = 0

def increment_counter_unsynchronized():
    global global_counter
    for _ in range(100_000):
        temp = global_counter
        temp += 1
        global_counter = temp

if __name__ == "__main__":
    print("Running Unsynchronized Multithreaded Program...")

    threads = []
    for _ in range(2): # Dua thread
        thread = threading.Thread(target=
increment_counter_unsynchronized)
        threads.append(thread)
        thread.start()

    for thread in threads:
        thread.join()

    print(f"Final global counter (unsynchronized): {global_counter}")
)
```

- b) Jalankan program 5 kali, simpan dan amati hasilnya (screenshot output).



```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
(base) amaheeng@Farhans-MacBook-Pro UAS % /usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/no_sync.py"
Running Unsynchronized Multithreaded Program...
Final global counter (unsynchronized): 200000
/usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/no_sync.py"
(base) amaheeng@Farhans-MacBook-Pro UAS % /usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/no_sync.py"
Running Unsynchronized Multithreaded Program...
Final global counter (unsynchronized): 200000
/usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/no_sync.py"
(base) amaheeng@Farhans-MacBook-Pro UAS % /usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/no_sync.py"
Running Unsynchronized Multithreaded Program...
Final global counter (unsynchronized): 200000
/usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/no_sync.py"
(base) amaheeng@Farhans-MacBook-Pro UAS % /usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/no_sync.py"
Running Unsynchronized Multithreaded Program...
Final global counter (unsynchronized): 200000
/usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/no_sync.py"
(base) amaheeng@Farhans-MacBook-Pro UAS % /usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/no_sync.py"
Running Unsynchronized Multithreaded Program...
Final global counter (unsynchronized): 200000
```

- c) Tambahkan Lock() untuk menghindari race condition. Jalankan kembali dan bandingkan hasilnya (screenshot output).



```
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
(base) amaheeng@Farhans-MacBook-Pro UAS % /usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/sync_lock.py"
Running Synchronized Multithreaded Program...
Final global counter (synchronized): 200000
(base) amaheeng@Farhans-MacBook-Pro UAS % /usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/sync_lock.py"
Running Synchronized Multithreaded Program...
Final global counter (synchronized): 200000
(base) amaheeng@Farhans-MacBook-Pro UAS % /usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/sync_lock.py"
Running Synchronized Multithreaded Program...
Final global counter (synchronized): 200000
(base) amaheeng@Farhans-MacBook-Pro UAS % /usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/sync_lock.py"
Running Synchronized Multithreaded Program...
Final global counter (synchronized): 200000
(base) amaheeng@Farhans-MacBook-Pro UAS % /usr/local/bin/python3 "/Users/amaheeng/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/KOMPAR/UAS/code/2/sync_lock.py"
Running Synchronized Multithreaded Program...
Final global counter (synchronized): 200000
```

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

d) Analisis perbedaan hasil dan penyebabnya.

Dalam percobaan saya, baik program tanpa sinkronisasi maupun dengan penggunaan Lock() sama-sama menghasilkan nilai akhir global counter yang konsisten, yaitu 200.000. Padahal secara teori, program tanpa Lock() seharusnya rawan mengalami race condition yang biasanya menghasilkan data yang tidak konsisten (kurang dari 200.000).

Secara teknis, operasi `global_counter += 1` bukanlah operasi atomik karena terdiri dari beberapa langkah (baca, modifikasi, dan tulis). Pada sistem multithreaded, seharusnya ada risiko dua thread bisa membaca nilai yang sama dan menulis hasil yang salah akibat "lost update".

Namun, hasil saya tetap konsisten kemungkinan karena:

- *Context switching* di sistem saya tidak terjadi di waktu yang kritis.
- Beban program cukup kecil, sehingga satu thread sering menyelesaikan tugasnya sebelum thread lain mengganggu.
- Hardware atau sistem penjadwal thread yang saya gunakan mungkin memberikan eksekusi yang cukup "bersih" dalam konteks sederhana ini.

Meskipun hasilnya kebetulan konsisten, secara prinsip program tanpa Lock() tetap tidak aman. Penambahan `threading.Lock()` sangat penting untuk memastikan hanya satu thread yang bisa mengakses dan memodifikasi variabel global pada satu waktu. Penggunaan Lock() membuat proses inkrementasi jadi aman dan mencegah race condition di sistem manapun.

Jadi, meskipun hasil percobaan saya terlihat baik tanpa Lock(), praktik terbaik tetap harus menggunakan Lock() untuk menjaga keamanan dan konsistensi data dalam pemrograman multithreaded.

e) Referensi:

1. Cari 2 referensi akademik yang menjelaskan race condition dan sinkronisasi

- a. Corbet, J., Kroah-Hartman, G., & Rubini, A. (2005). *Linux Device Drivers*, 3rd Edition. O'Reilly. ISBN: 0-596-00590-3.
- b. Blanton, E., Alphonse, C., & Mikida, E. (2024). *CSE 220: Systems Programming – "Races and Synchronization"* (slide). University at Buffalo

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

2. Cantumkan kutipan langsung yang relevan dari masing-masing referensi.

- **Kutipan tentang *Race Condition* (dari *Linux Device Drivers*):**

"This sequence of events is a demonstration of a race condition. Race conditions are a result of uncontrolled access to shared data. When the wrong access pattern happens, something unexpected results." (Chapter 5, Page 107)

- **Kutipan tentang Sinkronisasi (Locking/Mutual Exclusion - dari *Linux Device Drivers*):**

"The usual technique for access management is called locking or mutual exclusion—making sure that only one thread of execution can manipulate a shared resource at any time." (Chapter 5, Page 108)

"When semaphores are used for mutual exclusion—keeping multiple processes from running within a critical section simultaneously—their value will be initially set to 1. Such a semaphore can be held only by a single process or thread at any given time. A semaphore used in this mode is sometimes called a mutex, which is, of course, an abbreviation for 'mutual exclusion.'" (Chapter 5, Page 110)

3. Jelaskan bagaimana bagian tersebut mendukung analisis hasil eksperimen Anda

Kutipan dari *Linux Device Drivers* Bab 5 menjelaskan bahwa "*Race conditions are a result of uncontrolled access to shared data.*" Ini mendukung analisis saya bahwa kode tanpa Lock() secara teori tetap berisiko race condition, meskipun hasil eksperimen saya selalu konsisten di 200.000. Artinya, akses tanpa kontrol tetap bisa berpotensi menyebabkan hasil tidak akurat di kondisi atau sistem yang berbeda.

Kutipan berikutnya menjelaskan bahwa **locking atau mutual exclusion adalah solusi untuk mencegah race condition**, yaitu dengan memastikan hanya satu thread yang bisa mengakses data bersama pada satu waktu. Penggunaan `threading.Lock()` pada eksperimen saya membuktikan hal ini,

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

karena lock menjaga agar proses penambahan counter dilakukan secara aman dan hasilnya selalu akurat.

Jadi, meskipun hasil eksperimen saya terlihat konsisten tanpa lock, secara prinsip penggunaan lock tetap penting untuk menjaga keakuratan dan keamanan program multithreaded.

3. (30 Poin)

Sebuah sistem memproses data dari tiga sensor dengan interval sebagai berikut:

Sensor	Interval
Kamera	100 ms
LIDAR	50 ms
GPS	1 detik

a) Tentukan pendekatan paralelisme paling sesuai (multithreading atau multiprocessing) dan jelaskan alasannya.

Pendekatan paralelisme yang paling sesuai untuk kasus ini adalah multithreading. Alasannya adalah karena tugas dari setiap sensor pada simulasi ini termasuk I/O-bound, karena sensor hanya menunggu sesuai interval waktu dengan menggunakan fungsi `time.sleep()`. Pada kondisi seperti ini, Python akan melepas *Global Interpreter Lock* (GIL), sehingga thread lain tetap bisa berjalan meskipun satu thread sedang menunggu.

Multithreading menjadi pilihan yang tepat karena lebih ringan dan efisien untuk menangani tugas-tugas I/O seperti ini. Jika menggunakan multiprocessing, program tetap bisa berjalan, tetapi akan menimbulkan overhead yang lebih besar karena harus membuat dan mengelola proses yang terpisah, padahal sebenarnya tidak dibutuhkan untuk kasus sederhana seperti ini.

Dengan multithreading, program dapat menjalankan simulasi antar sensor secara bergantian dengan lancar dan sesuai dengan interval waktu yang diinginkan. Jadi, multithreading adalah pendekatan yang paling efektif dan efisien untuk menyelesaikan permasalahan ini.

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

- b) Simulasikan sistem menggunakan Python selama 5 detik. Gunakan sleep() untuk mensimulasikan interval masing-masing sensor dan cetak waktu serta nama sensor tiap kali sensor kirim data(screenshot output).

```
import threading
import time
import datetime

SIMULATION_DURATION = 5

def sensor_task(name, interval_ms):
    interval_s = interval_ms / 1000.0
    start_time = time.monotonic()

    print(f"[{datetime.datetime.now().strftime('%H:%M:%S.%f')}[:-3]] {name} sensor started (interval: {interval_ms} ms).")

    next_send_time = start_time + interval_s

    while time.monotonic() - start_time < SIMULATION_DURATION:
        sleep_until = next_send_time
        time_to_sleep = sleep_until - time.monotonic()

        if time_to_sleep > 0:
            time.sleep(time_to_sleep)

        if time.monotonic() - start_time >= SIMULATION_DURATION:
            break

        current_real_time = datetime.datetime.now().strftime('%H:%M:%S.%f')[:-3]
        print(f"[{current_real_time}] {name} sensor sent data.")

        next_send_time += interval_s
        while next_send_time < time.monotonic():
            next_send_time += interval_s

        print(f"[{datetime.datetime.now().strftime('%H:%M:%S.%f')}[:-3]] {name} sensor finished.")

if __name__ == "__main__":
    print("--- Starting Sensor Simulation ---")

    threads = []

    camera_thread = threading.Thread(target=sensor_task, args=("Kamera", 100))
    threads.append(camera_thread)

    lidar_thread = threading.Thread(target=sensor_task, args=("LIDAR", 50))
    threads.append(lidar_thread)

    gps_thread = threading.Thread(target=sensor_task, args=("GPS", 1000))
    # 1 detik = 1000 ms
    threads.append(gps_thread)

    for t in threads:
        t.start()

    for t in threads:
        t.join()

    print("--- Sensor Simulation Finished ---")
```

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

Hasil Output :

```
--- Starting Sensor Simulation ---
[22:09:02.097] Kamera sensor started (interval: 100 ms).
[22:09:02.097] LIDAR sensor started (interval: 50 ms).
[22:09:02.098] GPS sensor started (interval: 1000 ms).
[22:09:02.148] LIDAR sensor sent data.
[22:09:02.200] Kamera sensor sent data.
[22:09:02.200] LIDAR sensor sent data.
[22:09:02.249] LIDAR sensor sent data.
[22:09:02.302] Kamera sensor sent data.
[22:09:02.302] LIDAR sensor sent data.
[22:09:02.350] LIDAR sensor sent data.
[22:09:02.398] Kamera sensor sent data.
[22:09:02.401] LIDAR sensor sent data.
[22:09:02.451] LIDAR sensor sent data.
[22:09:02.500] Kamera sensor sent data.
[22:09:02.500] LIDAR sensor sent data.
[22:09:02.551] LIDAR sensor sent data.
[22:09:02.602] Kamera sensor sent data.
[22:09:02.602] LIDAR sensor sent data.
[22:09:02.651] LIDAR sensor sent data.
[22:09:02.701] Kamera sensor sent data.
[22:09:02.701] LIDAR sensor sent data.
[22:09:02.748] LIDAR sensor sent data.
[22:09:02.797] Kamera sensor sent data.
[22:09:02.798] LIDAR sensor sent data.
[22:09:02.850] LIDAR sensor sent data.
[22:09:02.902] LIDAR sensor sent data.
[22:09:02.902] Kamera sensor sent data.
[22:09:02.951] LIDAR sensor sent data.
[22:09:03.000] Kamera sensor sent data.
[22:09:03.000] LIDAR sensor sent data.
[22:09:03.051] LIDAR sensor sent data.
[22:09:03.100] LIDAR sensor sent data.
[22:09:03.100] Kamera sensor sent data.
[22:09:03.101] GPS sensor sent data.
[22:09:03.150] LIDAR sensor sent data.
[22:09:03.200] Kamera sensor sent data.
[22:09:03.200] LIDAR sensor sent data.
[22:09:03.252] LIDAR sensor sent data.
[22:09:03.303] Kamera sensor sent data.
[22:09:03.303] LIDAR sensor sent data.
[22:09:03.351] LIDAR sensor sent data.
[22:09:03.401] LIDAR sensor sent data.
[22:09:03.401] Kamera sensor sent data.
[22:09:03.450] LIDAR sensor sent data.
[22:09:03.499] Kamera sensor sent data.
[22:09:03.500] LIDAR sensor sent data.
[22:09:03.549] LIDAR sensor sent data.
[22:09:03.601] Kamera sensor sent data.
[22:09:03.601] LIDAR sensor sent data.
[22:09:03.651] LIDAR sensor sent data.
[22:09:03.701] Kamera sensor sent data.
[22:09:03.701] LIDAR sensor sent data.
[22:09:03.748] LIDAR sensor sent data.
[22:09:03.797] Kamera sensor sent data.
[22:09:03.798] LIDAR sensor sent data.
[22:09:03.851] LIDAR sensor sent data.
[22:09:03.898] Kamera sensor sent data.
[22:09:03.899] LIDAR sensor sent data.
[22:09:03.951] LIDAR sensor sent data.
[22:09:04.002] Kamera sensor sent data.
[22:09:04.002] LIDAR sensor sent data.
[22:09:04.052] LIDAR sensor sent data.
[22:09:04.098] Kamera sensor sent data.
[22:09:04.098] LIDAR sensor sent data.
[22:09:04.099] GPS sensor sent data.
```

```
[22:09:05.101] Kamera sensor sent data.
[22:09:05.101] LIDAR sensor sent data.
[22:09:05.102] GPS sensor sent data.
[22:09:05.152] LIDAR sensor sent data.
[22:09:05.200] Kamera sensor sent data.
[22:09:05.201] LIDAR sensor sent data.
[22:09:05.249] LIDAR sensor sent data.
[22:09:05.302] Kamera sensor sent data.
[22:09:05.303] LIDAR sensor sent data.
[22:09:05.348] LIDAR sensor sent data.
[22:09:05.398] Kamera sensor sent data.
[22:09:05.402] LIDAR sensor sent data.
[22:09:05.448] LIDAR sensor sent data.
[22:09:05.499] Kamera sensor sent data.
[22:09:05.499] LIDAR sensor sent data.
[22:09:05.550] LIDAR sensor sent data.
[22:09:05.601] Kamera sensor sent data.
[22:09:05.601] LIDAR sensor sent data.
[22:09:05.652] LIDAR sensor sent data.
[22:09:05.702] LIDAR sensor sent data.
[22:09:05.703] Kamera sensor sent data.
[22:09:05.749] LIDAR sensor sent data.
[22:09:05.799] Kamera sensor sent data.
[22:09:05.799] LIDAR sensor sent data.
[22:09:05.848] LIDAR sensor sent data.
[22:09:05.900] Kamera sensor sent data.
[22:09:05.900] LIDAR sensor sent data.
[22:09:05.950] LIDAR sensor sent data.
[22:09:06.001] Kamera sensor sent data.
[22:09:06.001] LIDAR sensor sent data.
[22:09:06.049] LIDAR sensor sent data.
[22:09:06.098] Kamera sensor sent data.
[22:09:06.098] GPS sensor sent data.
[22:09:06.098] LIDAR sensor sent data.
[22:09:06.148] LIDAR sensor sent data.
[22:09:06.201] Kamera sensor sent data.
[22:09:06.201] LIDAR sensor sent data.
[22:09:06.251] LIDAR sensor sent data.
[22:09:06.298] Kamera sensor sent data.
[22:09:06.302] LIDAR sensor sent data.
[22:09:06.352] LIDAR sensor sent data.
[22:09:06.397] Kamera sensor sent data.
[22:09:06.398] LIDAR sensor sent data.
[22:09:06.450] LIDAR sensor sent data.
[22:09:06.501] Kamera sensor sent data.
[22:09:06.502] LIDAR sensor sent data.
[22:09:06.550] LIDAR sensor sent data.
[22:09:06.602] Kamera sensor sent data.
[22:09:06.603] LIDAR sensor sent data.
[22:09:06.649] LIDAR sensor sent data.
[22:09:06.701] Kamera sensor sent data.
[22:09:06.701] LIDAR sensor sent data.
[22:09:06.748] LIDAR sensor sent data.
[22:09:06.800] Kamera sensor sent data.
[22:09:06.800] LIDAR sensor sent data.
[22:09:06.851] LIDAR sensor sent data.
[22:09:06.900] LIDAR sensor sent data.
[22:09:06.900] Kamera sensor sent data.
[22:09:06.952] LIDAR sensor sent data.
[22:09:07.001] Kamera sensor sent data.
[22:09:07.001] LIDAR sensor sent data.
[22:09:07.048] LIDAR sensor sent data.
[22:09:07.100] Kamera sensor finished.
[22:09:07.100] LIDAR sensor finished.
[22:09:07.100] GPS sensor finished.
--- Sensor Simulation Finished ---
```

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

c) Jelaskan hasil pengamatan Anda:

1. Apakah data terkirim sesuai interval?

Berdasarkan output yang saya peroleh, data terkirim cukup sesuai dengan interval yang sudah ditentukan.

- Sensor LIDAR (50 ms): Mulai pada pukul 22:09:02.097 dan mengirim data hampir setiap 50 ms.
- Sensor Kamera (100 ms): Mulai pada pukul 22:09:02.097 dan mengirim data sekitar setiap 100 ms.
- Sensor GPS (1000 ms): Mulai pada pukul 22:09:02.098 dan mengirim data kira-kira setiap 1 detik.

Dari hasil ini dapat disimpulkan bahwa pengiriman data berjalan sesuai interval yang diinginkan dan semua thread dapat berjalan secara bersamaan tanpa hambatan yang berarti. Perbedaan waktu beberapa milidetik masih dapat diterima dalam simulasi seperti ini.

2. Apakah terjadi keterlambatan/bottleneck? Mengapa?

Tidak terjadi keterlambatan atau bottleneck yang signifikan pada percobaan ini karena setiap thread hanya menjalankan tugas I/O-bound, yaitu menunggu menggunakan `time.sleep()`. Pada saat thread berada dalam kondisi sleep, Python melepaskan Global Interpreter Lock (GIL), sehingga thread lain tetap bisa berjalan tanpa harus menunggu. Hal ini memungkinkan ketiga thread sensor berjalan bersamaan dan mengirim data sesuai dengan interval yang ditentukan. Jika ada sedikit deviasi waktu, hal tersebut lebih disebabkan oleh proses penjadwalan thread oleh sistem operasi dan keterbatasan akurasi timer, bukan karena hambatan GIL. GIL baru akan menjadi masalah jika tugas yang dijalankan bersifat CPU-bound, seperti perhitungan berat, karena dalam kondisi tersebut Python hanya mengizinkan satu thread aktif di CPU pada satu waktu.

d) Referensi:

1. Sertakan 2 referensi teknis atau akademik yang mendukung pendekatan Anda.

- Piñeiro, C. and Pichel, J.C., (2024). *OMP4Py: a pure Python implementation of OpenMP*. arXiv preprint arXiv:2411.14887.

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

- Dokumentasi resmi Python tentang modul threading:
[Python docs.python.org](https://docs.python.org)
2. Tuliskan bagian spesifik dari masing-masing referensi yang relevan untuk mendukung analisis Anda.
- Kutipan dari OMP4Py (ArXiv 2024) menyatakan bahwa **“the GIL is released when a thread performs I/O operations, allowing other threads to execute Python code”** (Piñeiro et al., 2024, Hal. 3)
 - Dokumentasi resmi Python tentang modul threading juga menegaskan bahwa meski GIL membatasi eksekusi paralel pada kode CPU-bound, **“threading is still an appropriate model if you want to run multiple I/O-bound tasks simultaneously”**
3. Jelaskan bagaimana bagian tersebut mendukung analisis hasil eksperimen Anda

Kutipan dari OMP4Py mendukung hasil eksperimen saya karena simulasi sensor yang dijalankan memang lebih banyak melakukan operasi I/O, seperti `time.sleep()` untuk menunggu interval. Saat thread sensor sedang menunggu, GIL otomatis dilepaskan, jadi thread lain bisa langsung jalan tanpa harus menunggu. Ini menjelaskan kenapa ketiga sensor bisa berjalan bersamaan dan mengirim data tepat waktu tanpa bottleneck yang berarti, seperti yang saya lihat di hasil percobaan.

Selain itu, dokumentasi Python tentang threading juga mendukung alasan kenapa saya memilih multithreading untuk kasus ini. Python menjelaskan bahwa multithreading memang cocok untuk tugas I/O-bound karena saat thread sedang menunggu, GIL dilepas, jadi thread lain tetap bisa aktif. Ini sesuai dengan yang saya amati, di mana program tetap berjalan lancar dan tiap sensor mengirim data sesuai interval tanpa terhalang.

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

4. (30 Poin)

a) Buat program Python untuk mencari bilangan prima antara 1 dan 5.000.000.

```
import math
import time
from multiprocessing import Pool, cpu_count
import os

# memeriksa bilangan prima
def is_prime(n):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

# mencari bilangan prima
def find_primes_in_range(start, end):
    primes = []
    for num in range(start, end + 1):
        if is_prime(num):
            primes.append(num)
    return primes

# bilangan prima paralel
def run_prime_finder_parallel(num_workers):
    MAX_NUMBER = 5_000_000

    ranges = []

    step = MAX_NUMBER // num_workers
    for i in range(num_workers):
        start = i * step + 1
        end = (i + 1) * step
        if i <= num_workers - 1:
            end = MAX_NUMBER
        ranges.append((start, end))

    print(f"\n--- Running with {num_workers} worker(s) ---")
    start_time = time.perf_counter()

    all_primes = []
    with Pool(num_workers) as pool:
        results = pool.starmap(find_primes_in_range, ranges)
```

```
# Menggabungkan hasil dari semua worker
for prime_list in results:
    all_primes.extend(prime_list)

end_time = time.perf_counter()
execution_time = end_time - start_time
total_primes = len(all_primes)

print(f"Total primes found: {total_primes}")
print(f"Execution time with {num_workers} worker(s): {execution_time:.4f} seconds")
return execution_time, total_primes

if __name__ == "__main__":
    available_cores = cpu_count()
    print(f"Number of CPU cores available: {available_cores}")

    worker_configs = [1, 2, 4, 8]

    if any(w > available_cores for w in worker_configs):
        print(f"Warning: Some worker configurations ({worker_configs}) exceed the number of available CPU cores ({available_cores}). Performance might not scale linearly due to context switching overhead or system resource contention.")

    results_table = {
        "Jumlah Worker": [],
        "Waktu Eksekusi (s)": [],
        "Speedup": [],
        "Efisiensi (%)": []
    }

    time_1_worker = 0.0

    for num_workers in worker_configs:
        exec_time, total_primes = run_prime_finder_parallel(num_workers)

        results_table["Jumlah Worker"].append(num_workers)
        results_table["Waktu Eksekusi (s)"].append(f"{exec_time:.4f}")

        if num_workers == 1:
            time_1_worker = exec_time
            results_table["Speedup"].append("1.00")
            results_table["Efisiensi (%)"].append("100.00")
        else:
            speedup = time_1_worker / exec_time
            efficiency = speedup / num_workers * 100
            results_table["Speedup"].append(f"{speedup:.2f}")
            results_table["Efisiensi (%)"].append(f"{efficiency:.2f}")

    print("\n--- Hasil Tabel Eksekusi ---")
    print(
        "| Jumlah Worker | Waktu Eksekusi (s) | Speedup | Efisiensi (%) |"
    )
    print(
        "|-----|-----|-----|-----|"
    )
    for i in range(len(results_table["Jumlah Worker"])):
        print(f"| {results_table['Jumlah Worker'][i]:<13} | {results_table['Waktu Eksekusi (s)'][i]:<18} | {results_table['Speedup'][i]:<7} | {results_table['Efisiensi (%)'][i]:<13} |")
```

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

b) Jalankan program menggunakan multiprocessing.Pool atau ProcessPoolExecutor dengan:

- 1, 2, 4, dan 8 worker(screenshot output tiap worker).

```
/usr/local/bin/python3 "/Users/amaheeen/Library/CloudStorage/OneDrive-UniversitasPertamina/#SEM 4/K0
(base) amaheeen@Farhans-MacBook-Pro UAS % /usr/local/bin/python3 "/Users/amaheeen/Library/CloudStora
Number of CPU cores available: 4
Warning: Some worker configurations ([1, 2, 4, 8]) exceed the number of available CPU cores (4). Per
ntion.

--- Running with 1 worker(s) ---
Total primes found: 348513
Execution time with 1 worker(s): 19.1866 seconds

--- Running with 2 worker(s) ---
Total primes found: 348513
Execution time with 2 worker(s): 12.3445 seconds

--- Running with 4 worker(s) ---
Total primes found: 348513
Execution time with 4 worker(s): 11.7909 seconds

--- Running with 8 worker(s) ---
Total primes found: 348513
Execution time with 8 worker(s): 11.5008 seconds

--- Hasil Tabel Eksekusi ---
| Jumlah Worker | Waktu Eksekusi (s) | Speedup | Efisiensi (%) |
|-----|-----|-----|-----|
| 1 | 19.1866 | 1.00 | 100.00 |
| 2 | 12.3445 | 1.55 | 77.71 |
| 4 | 11.7909 | 1.63 | 40.68 |
| 8 | 11.5008 | 1.67 | 20.85 |
(base) amaheeen@Farhans-MacBook-Pro UAS %
```

c) Ukur waktu eksekusi pada tiap konfigurasi, lalu isi tabel berikut:

Jumlah Worker	Waktu Eksekusi (s)	Speedup	Efisiensi (%)
1	19.1866	1.00	100.00
2	12.3445	1.55	77.71
4	11.7909	1.63	40.68
8	11.5008	1.67	20.85

$\text{Speedup} = \text{waktu_1_worker} / \text{waktu_n_worker}$

$\text{Efisiensi} = \text{speedup} / \text{jumlah_worker} \times 100\%$

d) Analisis:

1. Apakah performa meningkat sebanding dengan jumlah worker?

Performa program **tidak meningkat secara sebanding atau linear** dengan penambahan jumlah worker. Saat saya menggunakan 1 ke 2 worker, ada peningkatan yang cukup terasa, dengan speedup dari 1.00 menjadi 1.55 dan waktu eksekusi turun dari 19.1866 detik menjadi 12.3445 detik. Namun, saat saya lanjut ke 4 worker, peningkatannya mulai melambat, speedup hanya naik ke 1.63 dan waktu eksekusi turun sedikit menjadi 11.7909 detik. Bahkan ketika saya coba pakai 8 worker, perubahannya sangat kecil, speedup hanya naik tipis menjadi 1.67 dan waktu eksekusi hampir tidak beda, yaitu 11.5008 detik.

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

Selain itu, efisiensi paralelisme juga menurun cukup drastis seiring bertambahnya worker. Kalau 1 worker efisiensinya tentu 100%, tapi saat pakai 8 worker, efisiensinya tinggal 20.85%. Ini menunjukkan bahwa menambah worker lebih banyak sudah tidak efektif lagi karena **biaya koordinasi antar proses dan overhead sistem** menjadi lebih besar daripada keuntungan yang didapat dari pembagian tugasnya.

2. Adakah titik di mana penambahan worker menjadi tidak efisien? Mengapa?

Penambahan worker mulai tidak efisien setelah 2 worker, apalagi ketika jumlah worker melebihi jumlah core CPU fisik. Pada 4 core, performa sudah mendekati batas optimal karena setiap core sudah terisi. Saat worker ditambah menjadi 8, sistem harus sering melakukan context switching, sehingga justru menambah beban dan mengurangi efisiensi. Selain itu, Hukum Amdahl membatasi speedup maksimal karena masih ada bagian program yang berjalan secara sequential dan tidak bisa diparalelkan.

e) Referensi:

1. Temukan 2 referensi akademik yang membahas speedup dan efisiensi paralelisme.

- Schryen, G. (2022). *Modeling Speedup and Efficiency in Parallel Computing*. ar5iv.org. Dapat diakses melalui ar5iv.org
- Myllis, G., Tsimpiris, A., Aggelopoulos, S. and Vrana, V.G., 2025. *High-Performance Computing and Parallel Algorithms for Urban Water Demand Forecasting*. Algorithms, 18(4), p.182.

2. Kutip bagian halaman tertentu yang relevan dengan hasil eksperimen Anda.

Schryen mendefinisikan "speedup S sebagai rasio waktu eksekusi sekuensial T_s terhadap waktu eksekusi paralel T_p [$S = T_s / T_p$]... dan efisiensi E sebagai perbandingan '*efficiency relates speedup to the number of parallel PUs... and it is defined by $E(P) = S(P)/P$ ' (Schryen, 2022, ar5iv.org). Ia juga menekankan bahwa "jika faktor overhead dimasukkan, maka fungsi eksekusi, speedup, dan efisiensi tidak lagi monoton meningkat. Ia menyimpulkan '*speedup and efficiency functions**

UJIAN AKHIR SEMESTER

Nama : Farhan Kholid

NIM : 105223024

may become non-monotonous, ... [maka] perlu ditentukan jumlah optimal unit pemrosesan paralel (PPUs) untuk metrik seperti speedup dan efisiensi" (Schryen, 2022, [ar5iv.org](https://arxiv.org)).

Myllis dkk. menegaskan bahwa "parallel efficiency, defined as the ratio of speedup to the number of cores used, provides insight into how well computational resources are utilized" (Myllis et al., 2025, mdpi.com). Mereka juga mengamati bahwa dalam eksperimen mereka, "efisiensi paralelitas bervariasi – misalnya pengoptimalan model deep learning mencapai efisiensi $\approx 71\%$ sementara tahap deteksi outlier hanya $\approx 21\%$ (karena beban komputasi rendah dan sinkronisasi tinggi)" (Myllis et al., 2025, mdpi.com).

3. Jelaskan bagaimana bagian tersebut mendukung analisis hasil eksperimen Anda

Kutipan dari Schryen (2022) mendukung definisi speedup dan efisiensi yang saya gunakan, sekaligus menegaskan bahwa penambahan worker tidak selalu meningkatkan performa secara terus-menerus karena adanya overhead paralelisme. Hal ini sesuai dengan hasil eksperimen saya, di mana setelah melewati 4 worker, penambahan menjadi 8 worker tidak lagi memberi peningkatan signifikan dan efisiensinya justru turun drastis. Kutipan dari Myllis et al. (2025) juga memperkuat bahwa efisiensi paralel sangat dipengaruhi oleh overhead, terutama saat sumber daya CPU mulai terbagi secara berlebihan. Penurunan efisiensi dalam eksperimen saya, dari 100% menjadi 20.85%, sejalan dengan observasi mereka bahwa pada kondisi tertentu, efisiensi bisa turun karena manajemen proses dan beban sistem yang meningkat. Kedua referensi ini membantu menjelaskan secara teori dan praktik kenapa performa tidak selalu naik seiring bertambahnya worker.